Worksheet 3: Consistent Project Structure

Kosmas Hench
2025-05-27

Contents

In this Session, the focus is on the organization and structure of a reproducible project. In practice, this translates to the management of the project content within a single folder. Throughout the workshop, we are looking at reproducibility from different angles – here, I hope for it to become apparent that most of these concepts are working in the context of such a managed folder. In a way, the different tools should interlock and complement each other, each one bringing a separate feature to the mix that optimizes the reproducibility of the project as a whole.

1. Recap

The main point stressing this structured approach to a reproducible project structure is to provide some predictability. If others can easily navigate your code and data, they naturally also will have better chances to recreate your analysis. Beyond this, we are setting all our reproducibility tools to the same reference point (the project folder) and facilitate overall portability of the project. Finally, by separating input, coding instructions and analysis results, we make it easy to deposit each aspect to a dedicated archiving service/repository, while also enabling the re-assembly of the project from these separate parts.

1.1 setting up the basic structure

Once more, we start out by creating a brand-new, empty folder and then populate it with a set of subdirectories. These will be subfolders for the data, code and results that provide the main structure for the project.

```
# --- setting up the basic project structure ---
# create a skeleton of nested directories for a new project
mkdir -p project_folder/data
mkdir -p project_folder/code/R project_folder/code/py
mkdir -p project_folder/results
```

Next, we initiate version control so that we are able to use `git` from the very onset of our work within the project.

```
# navigte into the new folder and intiate version control
cd project_folder
git init
touch .gitignore
```

We will need to document the purpose and content of the project. Therfore, we already put a `readme.md` in place, to make sure that we don't forget to do so later

```
# create readme file for the project
echo -e "# My Study Name\n\n**Author:** Your Name\n\n" > readme.md
echo -e "Description of the project ... " >> readme.md
```

Since we will rely on some `R` coding for our analysis, we turn the project folder into at `Rstudio` projcet by placing a `.Rproj` file within the root folder of the project.

```
# initiate Rstudio project
echo "Version: 1.0

RestoreWorkspace: No
SaveWorkspace: No
AlwaysSaveHistory: Default

EnableCodeIndexing: Yes
UseSpacesForTab: Yes
NumSpacesForTab: 2
Encoding: UTF-8

RnwWeave: Sweave
LaTeX: pdfLaTeX" > project_name.Rproj
```

At this point we have established the skeleton of our project-to-be:

```
# intial state of the project folder
tree
 project_folder/
 ├── code
 │   ├── py
 │   └── R
 ├── data
 ├── project_name.Rproj
 ├── readme.md
 └── results
```

Given that the basic layout of the project is in place, we commit the initial state to version control as the starting point of our project's track record.

```
# commit the initial state of the project folder
git add .
git commit -m "project setup"
```

1.2 populating and using the project structure

Now, let's look at how we are going to use the project structure in practice.
For this, we'll do another small dummy analysis, starting by downloading a "mysterious" data set into our `data/` directory.

```
# downloading external data
cd data/
wget https://raw.githubusercontent.com/k-hench/repcoding_online_resources/refs/heads/main
    /data/dd_masked.tsv
cd ..
```

The data set appears rather generic, and it contains the three columns `x`, `y` and an `id`.

First lines of `dd_masked.tsv`:

id	x	y
1	75.1282	79.1026
2	65.1507052757	92.502633736799993
3	47.4421116354	98.1843018589
…	…	…

The initial exploration of this data set will happen using some `python` code. The script for this part is available the `external_resources` folder, located in your `$HOME` directory of your de.NBI instance. Naturally, we copy this script into our subdirectory `code/`, more specifically into `code/py/`.

```
# copy python script into the project
cp ~/external_resources/scripts/py/dd_explore.py code/py/
```

The script should run straight out of the box, when invoked from the "root" folder of our project:

```
# run python script
python code/py/dd_explore.py > results/python_env.log
```

Let's look into the `python` script to see what we are actually doing here.
The script starts with a comment, telling us how it is intended to be run. Then some required `python` packages are loaded.

dd_explore.py

```
# this python script should be run from the root of the project (project_folder)
# python ./code/py/dd_explore.py
import session_info   # to capture the computing environment
import pandas as pd   # to help organize data
import seaborn as sns # for plotting
```

Still in the "configuration" section of the script, the plotting theme is set.

dd_explore.py

```
# apply the default plotting theme
sns.set_theme(style = "ticks", font_scale = 0.85, rc={'figure.figsize':(5,5)})
```

Then, the actual analysis starts with reading in our downloaded data set. Note how the path is relative to the project's "root" folder, matching the directory from which the `python` script is intended to be run.

dd_explore.py

```
# load data from TSV file, first three lines are comments
data = pd.read_csv('data/dd_masked.tsv', delimiter = '\t')
```

Our "data exploration" is simply going to be to produce a scatter plot of the `x` versus `y` column.

dd_explore.py

```
# scatterplot of the data for a first inspection
g = sns.relplot(
    data = data,
    x = "x", y = "y", s = 2.5
)
```

We add a title to the figure, and export the figure to a `png` file. Again, note how the path of the output file is relative to the project "root" folder, this time dumping the figure into the `results/` subdirectory.

dd_explore.py

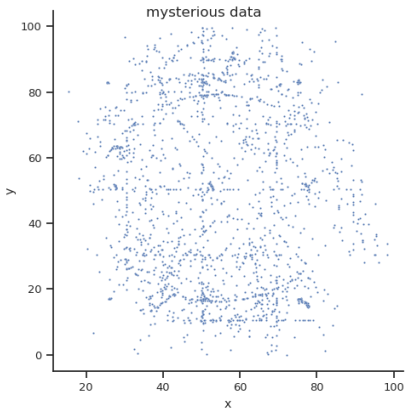
```
# add title to the plot
g.fig.suptitle("mysterious data", fontsize=12)
# save plot to file
g.fig.savefig("results/out.png")
```

Remembering our lessons from Section 1, we finish of the `python` script by capturing the used working environment (which we write into the log file `results/python_code.log`).

dd_explore.py

```
# Log session information
print("python packages used:\n---------- ---------- ----------")
session_info.show()
print("---------- ---------- ----------\n")
```

The plot we just created gives us a first glimpse of the data. However, it is not too illuminating:



scatter plot of the `data/dd_masked.tsv` data

So, it turns out to make sense of the data, we actually require some additional information. Like the `python` script before, we can find "`dd_key.tsv`" in the `external_resources` folder of your de.NBI account (in the subdirectory `external_resources/data`).
Obviously, this is a HUGE file (20Kb, *cough, cough*), which is why we are going to create a link to it instead of copying it, so that we do not need to duplicate the file.

```
40 # link external data into the project
41 cd data
42 ln -s ~/external_resources/data/dd_key.tsv ./
43 cd ..
```

We can see that this file shares the `id` column with `dd_maksed.tsv`, but it also contains a `row_number` and a `class` column:

First lines of `dd_key.tsv`:

id	row_number	class
1	86	A
2	1501	K
3	1560	K
...	...	...

Now, to resolve the mystery, we are going to combine the two data sets using some `R` scripting. You can find the required code at `external_resources/scripts/R/dd_resolve.R` and copy it into the project.

```
44 # copy R script into the project
45 cp ~/external_resources/scripts/R/dd_resolve.R code/R/
```

Again, the `R` script should run just like that. And similarly to the `python` script before, you can execute it from the project "root" folder.

```
46 # run R script from project_folder/code/R
47 Rscript code/R/dd_resolve.R
```

Let's try to follow the script to see how it completes the "analysis" while making use of the project structure.

Like always, we start the script by loading the required `R` packages.

dd_resolve.R

```
1 # importing required r packages
2 library(tidyverse) # for data handling & plotting
3 library(here) # to specify paths relative to .Rproj file
4 library(glue) # for error message formatting
```

The purpose of the `R` code is to demonstrate the functionality of the `here` package. To drive home that point, we are going to create some situations that are bound to fail. Here, we set up a wrapper function that will allow us to continue running the `R` script, even if a particular section fails and throws an error (which would typically abort the whole execution).

dd_resolve.R

```
# wrapper function to allow the script to continue
# also in case of errors during the file import
try_read_tsv ← \(file, ...) {
  tryCatch(
    {
      # case whithout errors
      read_tsv(file)
    },
    # case in which the data import creates an error
    error = function(e) {
      print(
        glue("Error while reading the file {file}:\n"),
        e$message
      )
    }
  )
}
```

Within the script, we are going to import the data from `dd_masked.tsv` two times:
once the standard `R` way, and once more utilizing the `here` package.

dd_resolve.R

```
# trying to read in data, assuming the working directory is `project_folder`
data_plain_path ← try_read_tsv("data/dd_masked.tsv")
```

As we will see, the approach using `here` is more robust, so we are also going to use that to import the second data set from `dd_key.tsv`.

dd_resolve.R

```
# reading in data while setting the working directory explicitly to `project_folder`
# (location of the .Rproj file)
# importing the mysterious "masked" data set
data_masked ← read_tsv(here("data/dd_masked.tsv"))
# importing the key to the "masked" data set
data_key ← read_tsv(here("data/dd_key.tsv"))
```

To combine the information from the two data sets, we join them based on their shared `id` column.

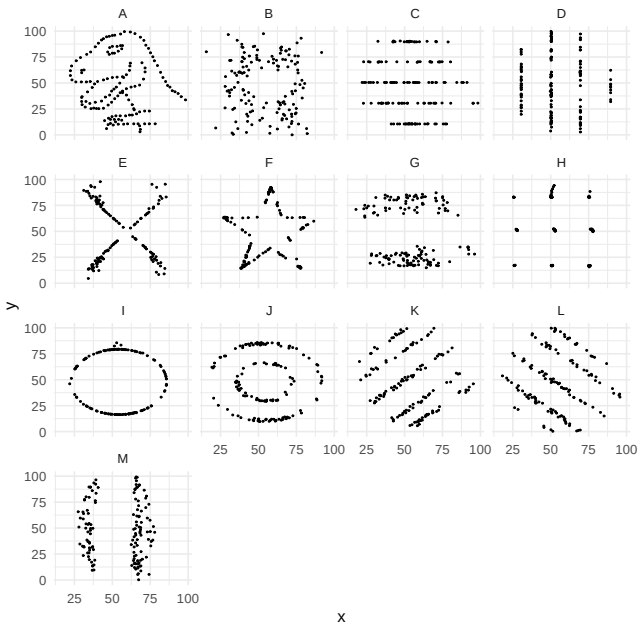
dd_resolve.R

```
# combine the key and the original data
data ← data_masked ▷
  left_join(data_key) ▷
  arrange(row_number)
```

Now we can repeat the scatter plot exercise — but this time we are going to seperate the data set based on their assigned `class`.

dd_resolve.R

```
# plotting the data set for which the import
# should have worked in any case
p ← data ▷
  ggplot(aes(x = x, y = y)) +
  geom_point(size = 0.25) +
  facet_wrap(class ~ .) +
  theme_minimal()
```



scatter plot by `class`

Eureka, the mystery is resolved: apparently we were looking at a mixed data set that was composed of several differently structured subsets.

To double down on the utility of the `here` package, we are now going to export that figure two times (again once the standard way, and once using `here`).

The first (standard) export is only for educational purposes, so we just going to dump it in the current working directory.

dd_resolve.R

```
47 # export the plot with a generic file name
48 ggsave(
49   filename = "plot_standard.pdf",
50   width = 6,
51   height = 6,
52   device = cairo_pdf
53 )
```

In contrast, the export using `here` is the encuraged approach. So in this case we specify the appropriate export location within the `results/` folder.

dd_resolve.R

```
55 # export the plot, while securing the
56 # output path within the project_folder
57 ggsave(
58   filename = here("results/plot_here.pdf"),
59   width = 6,
60   height = 6,
61   device = cairo_pdf
62 )
```

Just as another reminder of SESSION 1 we are displaying the `R` environment at the end of the script. We could log that in a similar way that we did for the `python` script.
But this time we skip that option (we are going to look at another neat feature of `Rstudio` projects in a minute instead...).

dd_resolve.R

```
64 # log session information
```

```
65 cat("R packages used:\n")
66 cat(str_c("run on the ", Sys.Date()))
67 cat("\n--- --- --- --- ---\n")
68 sessionInfo()
```

So what was all that fuss about using `here` about?

To illustrate this, let's launch the same `R` script from a different directory. This will change the working directory of the that `R` session (which is the launching directory by default).

We navigate into the subdirectory `code/R/` and re-run the script from there.

```
48 # run R script from project_folder/
49 cd code/R
50 Rscript dd_resolve.R
51 cd ../../
```

You should notice that the script still runs "properly", but there are two side effects: somewhere in the output, we will find the error message that the standard file import for `dd_masked.tsv` did not work this time (line 25 within the `R` script):

```
Error while reading the file data/dd_masked.tsv:
data/dd_masked.tsv does not exist in current working directory (/home/ubuntu/
    project_folder/code/R).
```

Also, both runs of the script exported a pdf file called `plot_standard.pdf`.
But, the first run dumped this file into the root folder, while the second run exported the file into `code/R`.
This is because the working directory was set to different locations in these two instances (by calling the script from these locations).

Yes, it would be possible to control for this by including `setwd()` at the beginning of the `R` script. But this would set the working directory to a specific location on whatever computer the script was written on. By using `here`, we can set the working directory relative to the `.Rproj` file, which makes it portable across different machines. That way we can make sure the `R` code always works relative to the "root" folder of our project.

Now, regarding the paper trail for the used `R` working environment (the used version of `R` and all the `R` packages). Instead of just logging all the version numbers into a log file, we are going to make use of the `renv` package. This will report all package version numbers, in a way that is more like a recipe than just a list of numbers. In fact, captured by `renv`, it will be possible to also automatically *restore* this environment (again with `renv`).

```
52 # used renv package to capture the current R working environment
53 R --slave -e 'renv::init(force = TRUE)'
```

Sweet, now that we are happy with the analysis and the setup, let's clean up the two files that were only created to make a point:

```
54 # cleanup - we only created these to make a point
55 rm plot_standard.pdf code/R/plot_standard.pdf
```

Finally, we can commit our clean analysis to version control.

```
56 # prevent plots from being logged
57 echo "*.png" >> .gitignore
58 echo "*.pdf" >> .gitignore
59 # commit the latest updates to version control
60 git add .
61 git status
62 git commit -m "mystery solved"
```

2. Outlook

While we are done with the coding for this Session, let's have a quick look how the content of the two remaining Sessions is going to be integrated into our established project structure.

The next topic is going to be the organization of an analysis into a "workflow". Specifically, we are going to be looking at `snakemake`, and how we can use it to automatize a complicated multi-step analysis in a clean way. The content of this will be sorted into a subdirectory of the `code` folder, specifically into `code/workflow`.

```
# --- outlook ---
mkdir -p code/workflow
touch code/workflow/snakefile
mkdir -p code/workflow/rules
```

And finally, in the last session, we will talk about how to capture working environments more generally (beyond what we just did with `renv`). Some aspect of this is going to be managed within `code/workflow/envs`

```
mkdir -p code/workflow/envs
```

So, an overall overview of our desired project structure will look something like this:

```
# final folder overview
tree -L 3
```

```
project_folder/
├── code/
│   ├── R/
│   │   └── dd_resolve.R
│   ├── py/
│   │   └── dd_explore.py
│   └── workflow/
│       ├── envs/
│       ├── rules/
│       └── snakefile
├── data/
│   ├── dd_key.tsv → /home/ubuntu/external_resources/data/dd_key.tsv
│   └── dd_masked.tsv
├── project_name.Rproj
├── readme.md
├── renv/
│   └── ...
├── renv.lock
└── results/
    ├── out.png
    ├── plot_here.pdf
    └── python_env.log
```